

Scalability and Future Plan

Phase 8 · Project Demonstration

Niranjan S R (Team Lead) · Pavithran P R · Srinivas S M · Sriram S

This document explains how the field-value-driven ACL pattern built for u_institution_details scales beyond a single hard-coded branch, and the enhancements planned to make it reusable across the institution. The current build proves the pattern for one branch (EEE); the items below generalise it without changing the core idea that access equals role plus field value.

Current Limitation to Address

Today the READ ACL is tied to one branch value (Branch is EEE) and one role (bb1). Every additional branch would need its own condition, role, and ACL, which does not scale as branches or tables grow.

How the Pattern Scales

Direction	Approach	Benefit
Per-user branch	Add a Branch field to the user record and match it in the ACL script (compare the record's Branch to the current user's Branch)	One ACL serves all branches; no per-branch role or condition
Dynamic query	Use a scripted before-query or a Query rule so each user only sees rows matching their branch	Filtering happens once, list and API stay consistent
More tables	Reuse the same role-plus-field pattern on other institution tables (attendance, fees, results)	Consistent record-level privacy across modules
Field-level ACLs	Add ACLs on individual fields (for example hide Phone Number from some roles)	Column-level privacy on top of row-level control
Automation	Assign bb roles and set the user Branch through data-lookup rules or a role-provisioning flow	Onboarding a branch user needs no manual ACL edits

Reference: Parameterised READ Logic (future form)

- Keep the admin bypass (gs.hasRole('admin') returns true).
- Replace the fixed bb1 branch with a comparison of the record's Branch to the signed-in user's Branch field.
- Result: a CSE user automatically sees only CSE records with the same single ACL that an EEE user uses.

Future Enhancements

- Data-driven roles: derive read/create/write/delete rights from a single branch attribute instead of four separate bb roles.
- Audit reporting: log denied access attempts to show the security stakeholders the control is active.

- Scoped application: package the table, roles, and ACLs into a scoped app for clean install on other PDIs or a production instance.
- Automated tests: background scripts that impersonate each persona and assert the expected record counts on every change.
- Delegated administration: let a branch coordinator manage only their branch's records without full admin.